TECHNICAL TRAINING · PART TWO

Coordination & Consensus

How independent services agree, hand off work and stay available when parts of the system are down or slow.

Continues Distributed System Design · Sessions 08–15
Caching, sagas, coordination and consensus in production systems

Vivekanand Pujar | @VivekanandPujar

SESSION MAP

Agenda

01 **Distributed caching**

A cache is a bet that stale is cheaper than slow

02 **Saga**

Once the transaction is gone, the rollback has to be designed by hand

03 **Choreography**

No conductor; each service reacts to what already happened

04 **Orchestrator**

One place that knows the whole plan, and the one place that can stall it

05 **Scatter and gather**

A fan-out request is only as fast as its slowest reply

06 **Replication and sharding**

Availability and scale are bought with different currencies

07 **Distributed consensus (RAFT)**

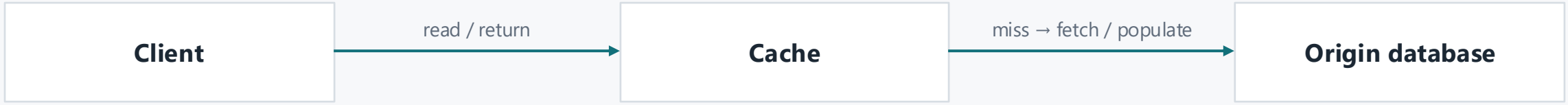
Trading a little availability for one agreed-upon truth

08 **Case studies**

DynamoDB, etcd and PostgreSQL MVCC, in production

SESSION 01

A cache is a bet that stale is cheaper than slow



Three ways to keep it correct

Read-through

Cache loads on miss, transparently; every request pays a latency tax on the first hit.

Write-through

Every write updates cache and database together; writes get slower so reads stay correct.

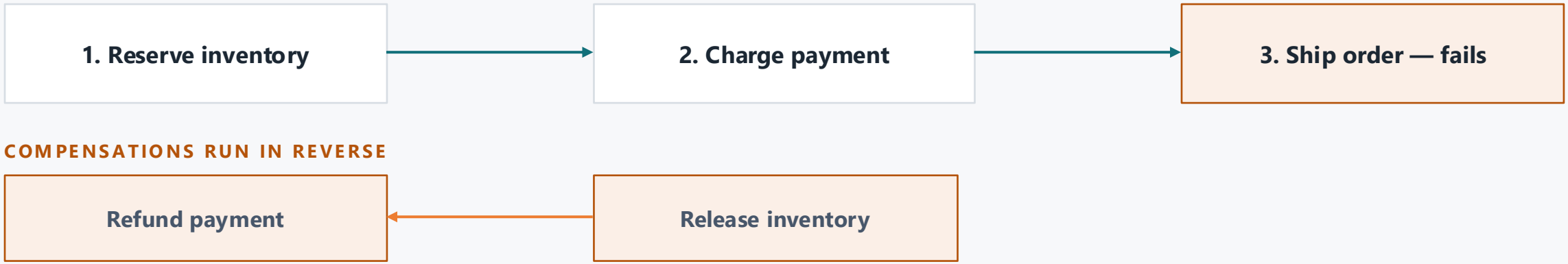
Write-behind

Writes land in cache first and drain to the database async; fast, but a crash can lose data.

Invalidation, not lookup, is where caching strategies fail.

SESSION 02

Once the transaction is gone, the rollback has to be designed by hand

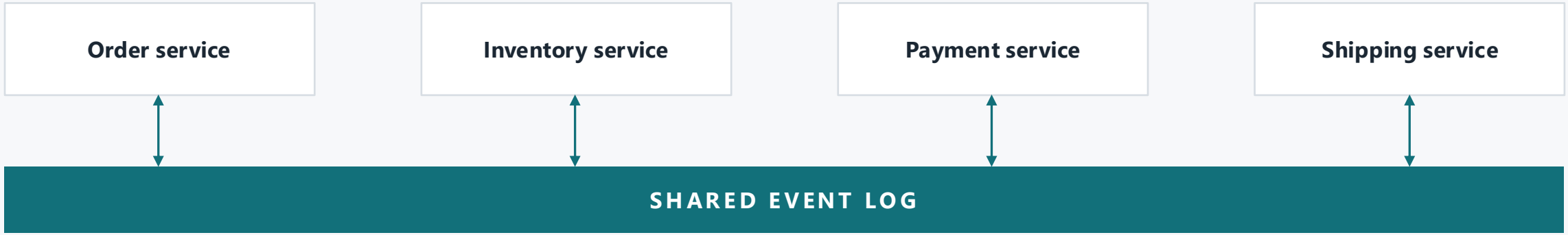


The saga replaces one atomic commit with a chain of local transactions

Each step commits on its own service; if a later step fails, the saga runs compensating actions for every step that already succeeded — an explicit undo, not a rollback.

SESSION 03

Choreography has no conductor; each service reacts to what already happened



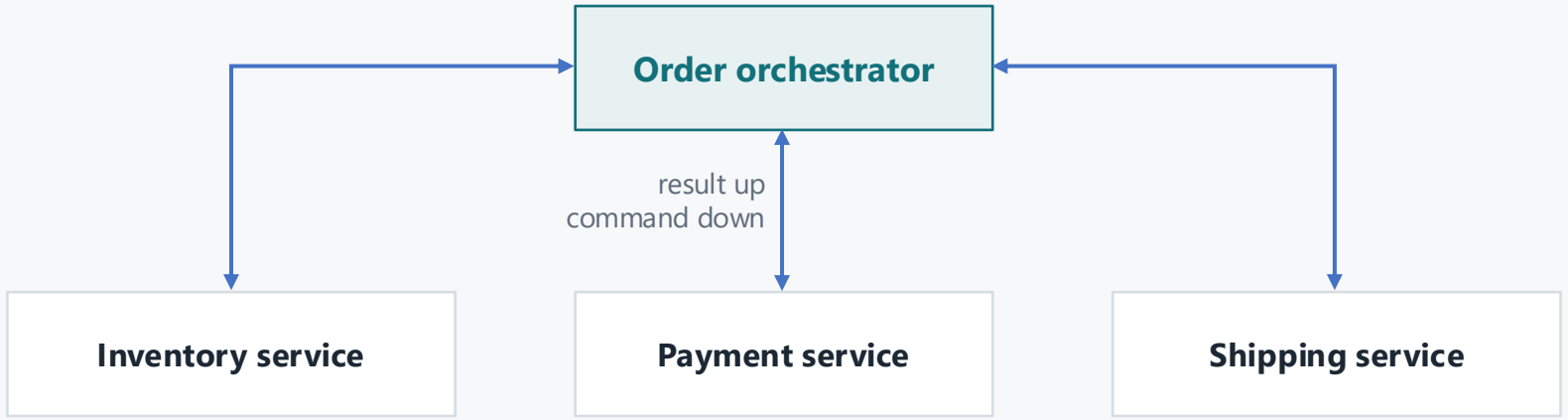
Each service publishes what it did and subscribes to what it needs

Inventory publishes “reserved”, payment reacts to it and publishes “charged”, shipping reacts to that. No service calls another directly, and no service holds the whole flow in its head.

Resilient to any one service being down; hard to see the flow that failed.

SESSION 04

One place knows the whole plan, and one place can stall it



What the orchestrator buys, and what it costs

BUYS

The whole workflow is readable in one place; retries, timeouts and compensation logic live in one service instead of being scattered across subscribers.

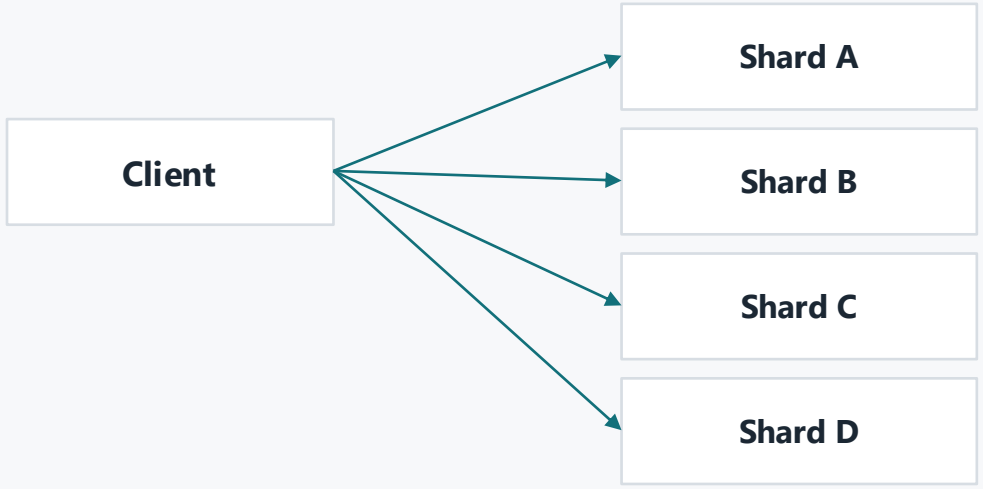
COSTS

Every step now depends on the orchestrator being reachable; its state store becomes the record of what is mid-flight — lose it, and no one knows what to resume.

Choreography trades a visible flow for no single owner; orchestration trades that back for a single dependency.

SESSION 05

A fan-out request is only as fast as its slowest reply



The tail, not the average, sets the deadline

- Latency is the slowest of N replies, not the mean of N — more shards, worse tail.
- Set a deadline and return with what has arrived; flag the rest as partial rather than blocking the caller.
- Hedge the request: fire a second copy at a different replica once the first passes a fixed threshold.

Every shard added to a scatter is another chance for the tail to get longer.

SESSION 06

Availability and scale are bought with different currencies

REPLICATION

Same data, many copies.

BUYS

Buys availability and read throughput — the system survives losing a node without losing the data.

COSTS

Costs write latency: every write must reach a leader or a quorum before it is durable, and followers can lag behind.

SHARDING

Different data, many partitions.

BUYS

Buys write throughput and storage that grows with the cluster instead of with one machine.

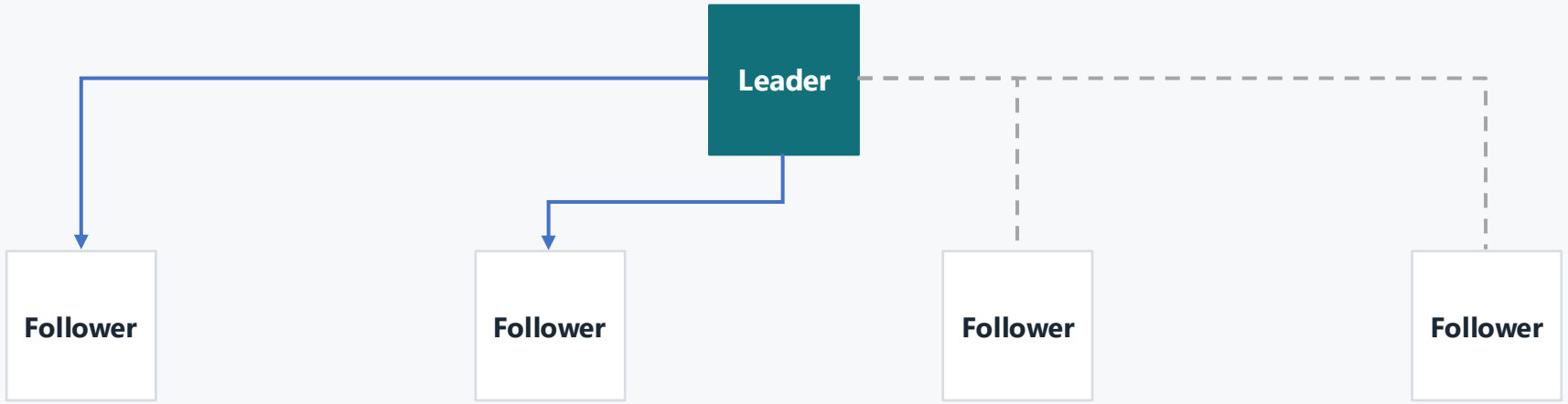
COSTS

Costs query simplicity: anything that spans shards is now a distributed query, and rebalancing moves live data.

Replication survives losing a machine. Sharding survives losing to growth. Most systems that scale need both — and most outages start by conflating them.

SESSION 07

Trading a little availability for one agreed-upon truth



log entry replicated · committed once 3 of 5 acknowledge

Three problems, one algorithm

Leader election

A randomized timeout so exactly one candidate usually wins a term, without two leaders active at once.

Log replication

The leader appends an entry and replicates it to followers, calling it committed once a majority hold it.

Safety

A candidate cannot become leader without already holding every entry the previous leader committed.

A write is durable at a majority, not at all nodes — which is exactly why the cluster can lose a minority and lose nothing.

SESSION 08

Walking one write through the log

1. **Client sends**
   SET x = 5 to the leader
2. **Leader appends**
   the entry to its own log, uncommitted
3. **Leader replicates**
   the entry to every follower
4. **Followers append**
   the entry and acknowledge it
5. **Majority acks → committed**
   applied to the state machine, client gets its reply

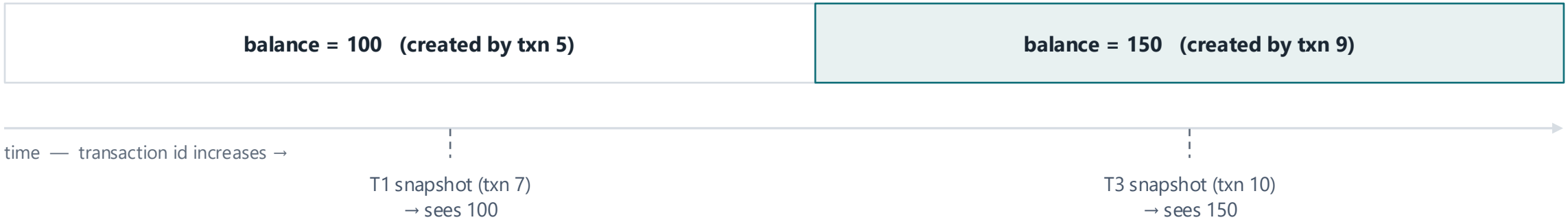
Why replicate before replying?

If the leader answered the client as soon as it wrote its own log, a crash one instant later would lose an acknowledged write. Raft makes the leader wait for a majority to hold the entry first — durability before reply, not after.

The client waits for one round trip to a majority, not to every node — that's the latency Raft actually costs you.

SESSION 09

A reader gets a snapshot; a writer gets a new row version



The rule, in three lines

- Every transaction gets a snapshot: the set of committed writes it may see, fixed the moment it starts.
- A write never overwrites in place — it inserts a new row version and marks the old one superseded.
- A reader just skips versions created after its snapshot, or removed before it. No lock required.

MVCC trades "block the reader" for "give the reader an older version" — the cost lands on disk space and VACUUM, not on query latency.

SESSION 10

Three engines, three different bets

System	Mechanism	Optimizes for	Costs
DynamoDB	Leaderless replication + consistent hashing; tunable read consistency	Availability and predictable low latency at any scale	Read-your-writes needs an explicit strong read; conflicting writes are resolved by the application
etcd	RAFT-replicated log; single elected leader per term	One agreed order of truth for cluster config and coordination	Writes need a majority quorum and stall during a leader-less window; writes don't scale horizontally
PostgreSQL (MVCC)	Every transaction reads a snapshot; writers create new row versions instead of locking	Readers never block writers, and writers never block readers	Old row versions accumulate until vacuumed; long transactions bloat storage and slow vacuum

Each engine picked one guarantee to be uncompromising about, and pushed the cost of the other two onto whoever operates it.

CLOSING

Six questions for the next design review

CACHING & WORKFLOW

01 Which endpoint can serve a stale read, and how stale before it's wrong?

02 If step three of five fails, who wrote the compensations for steps one and two?

03 Does this workflow need one service to know the whole plan, or can each step react to what already happened?

COORDINATION

04 When nine of ten replies are back, does the caller wait for the tenth or return with nine?

05 Is this shard boundary drawn by access pattern, or by convenience?

06 Which write here needs a majority of nodes to agree before it's durable — and which needs only one?

Bring the service that scares you most under partial failure. We'll trace where it caches, where it hands off work, and where it needs the cluster to agree.

Recommended reading: Ongaro & Ousterhout, "In Search of an Understandable Consensus Algorithm" (Raft); Kleppmann, Designing Data-Intensive Applications, ch. 5, 7, 9.